Decorator Pattern

The Decorator Pattern is a structural design pattern that enables you to dynamically add new functionalities to objects without changing their structure. This pattern generates a decorator class that wraps the original class and adds functionality, separating the class behaviors and promoting flexible and reusable design.

The Decorator Pattern is a great example of the Open/Closed Principle, which is one of the fundamental principles of object-oriented design. According to this principle, software entities (such as classes, modules, functions, etc.) should be closed for modification but open for extension.

How the Decorator Pattern Applies the Open/Closed Principle:

- **Closed for Modification**:

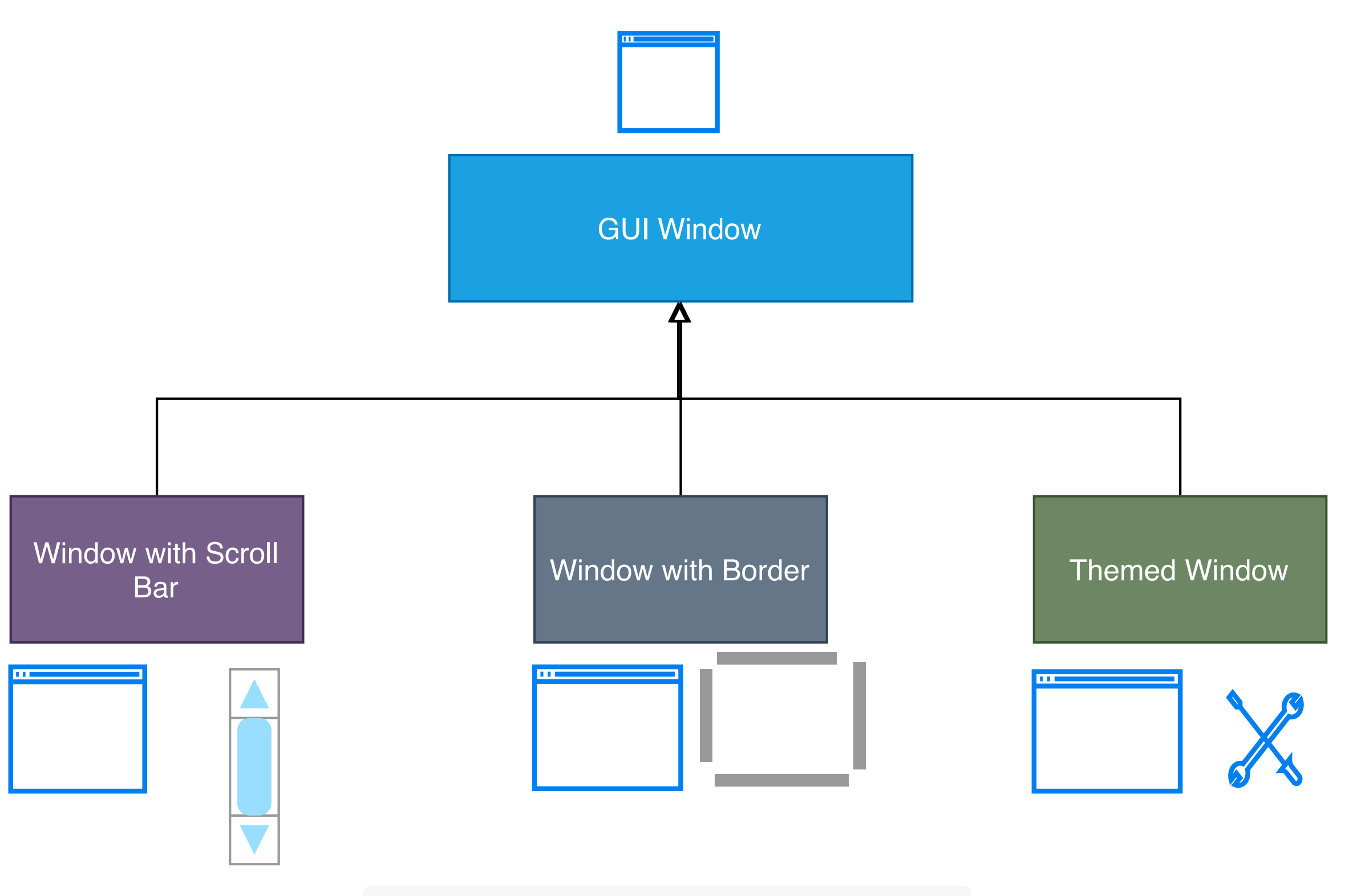
  In the Decorator Pattern, we don't modify the existing code to add some new functionality. Rather a new decorator class is generated that wraps the original class and adds new functionality. This means the original class code remains unchanged, adhering to the 'closed for modification' part of the principle.

- **Open for Extension**:

  The Decorator Pattern enables dynamically adding new functionalities. You can create new decorator classes that encapsulate the original class and add new behaviors. The ability to add new features or behaviors by adding new decorator classes demonstrates the 'open for extension' aspect of the principle.

Real-world Example

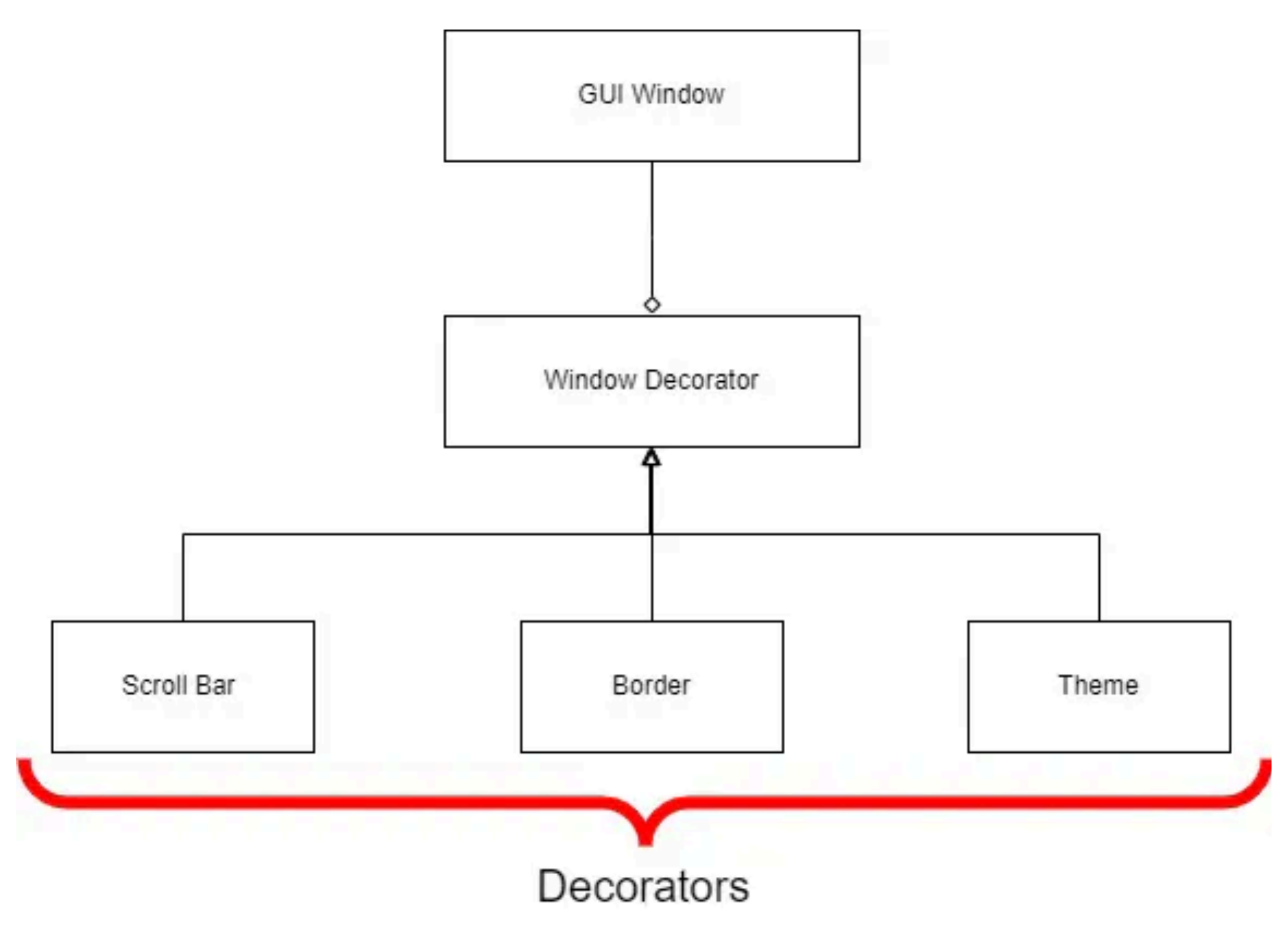
Consider a scenario where we have a basic `Window` class in GUI framework. After some time, we need to change that window and add some borders to it, then add a scroll bar to it. With more advancements, we need to add some themes to that window. Without the Decorator Pattern, this would lead to complex subclassing for every combination (like `WindowWithScrollbar`, `WindowWithBorder`, `ThemedWindow`, etc.).



Decorator Pattern - Problem Illustration

But, what can be the solution to this? Decorator patterns can jump in to help us!

With the use of decorator patterns, all these additional features will become decorators like `ScrollBarDecorator`, `BorderDecorator`, and `ThemeDecorator,` extending the `Window` class. We can add these decorators to the `Window` class dynamically.

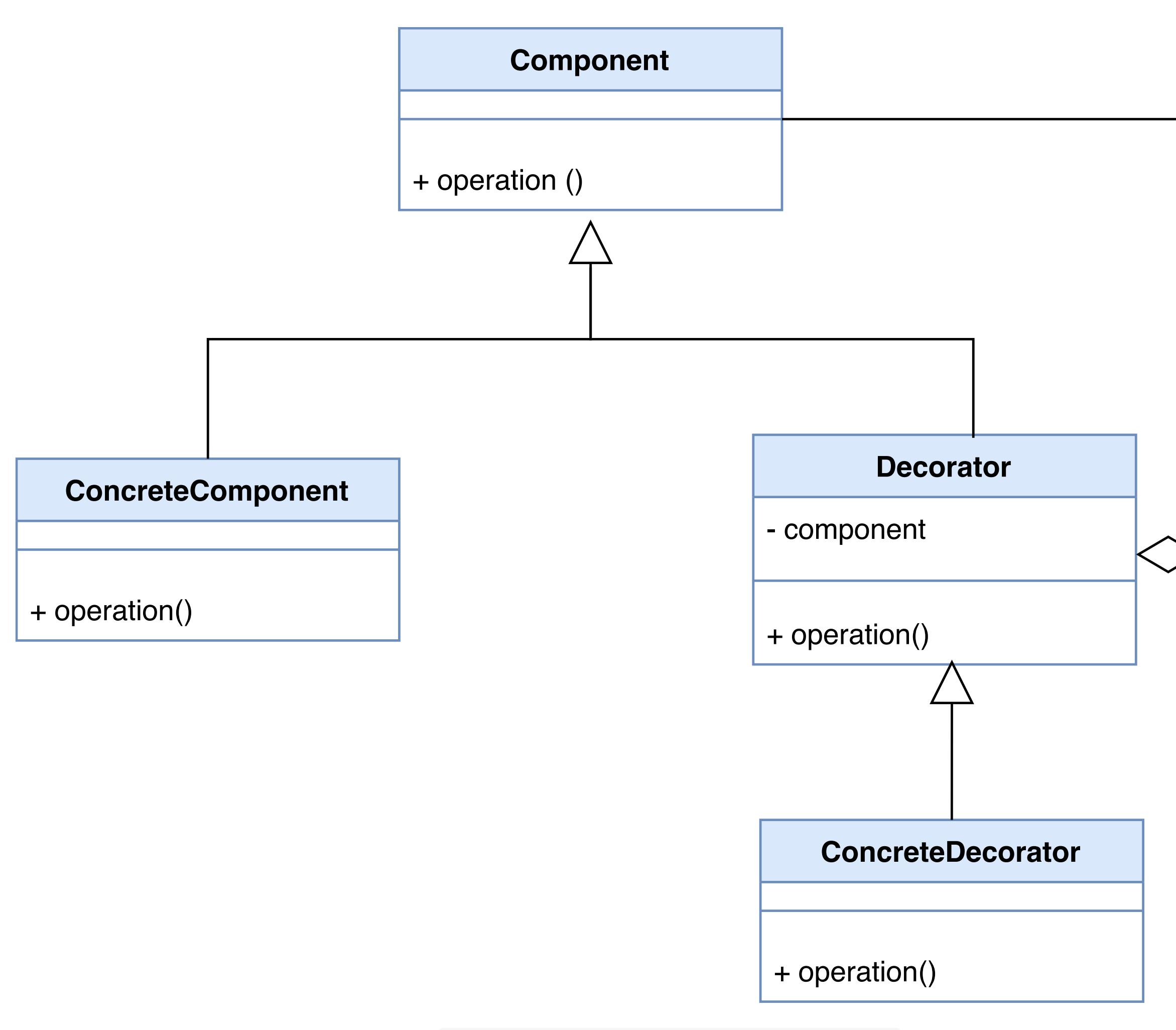


Decorator Pattern - Solution Illustration

Structure of Decorator Pattern

The main components of the Decorator Pattern include:

- **Component Interface**: This is the foundational interface for all pattern objects, defining the default behavior that decorators can add to.
- **Concrete Component**: A particular Component Interface implementation. This is the target to which additional responsibilities can be applied.
- **Decorator Interface**: An abstract class or interface that 'wraps' a component to give it extra features. It usually implements the same interface and stores a reference to an object representing a Component Interface.
- **Concrete Decorator**: These are the classes that give the decorated objects extra functions. They put the extra behaviors into practice and expanded the Decorator Interface.



Decorator Pattern - Class Diagram

Implementation of Decorator Pattern

Consider a Pizza creation application in which you can create pizzas with multiple toppings. Each pizza will have its own flavor and cost. This application can be implemented using the Decorator Pattern.

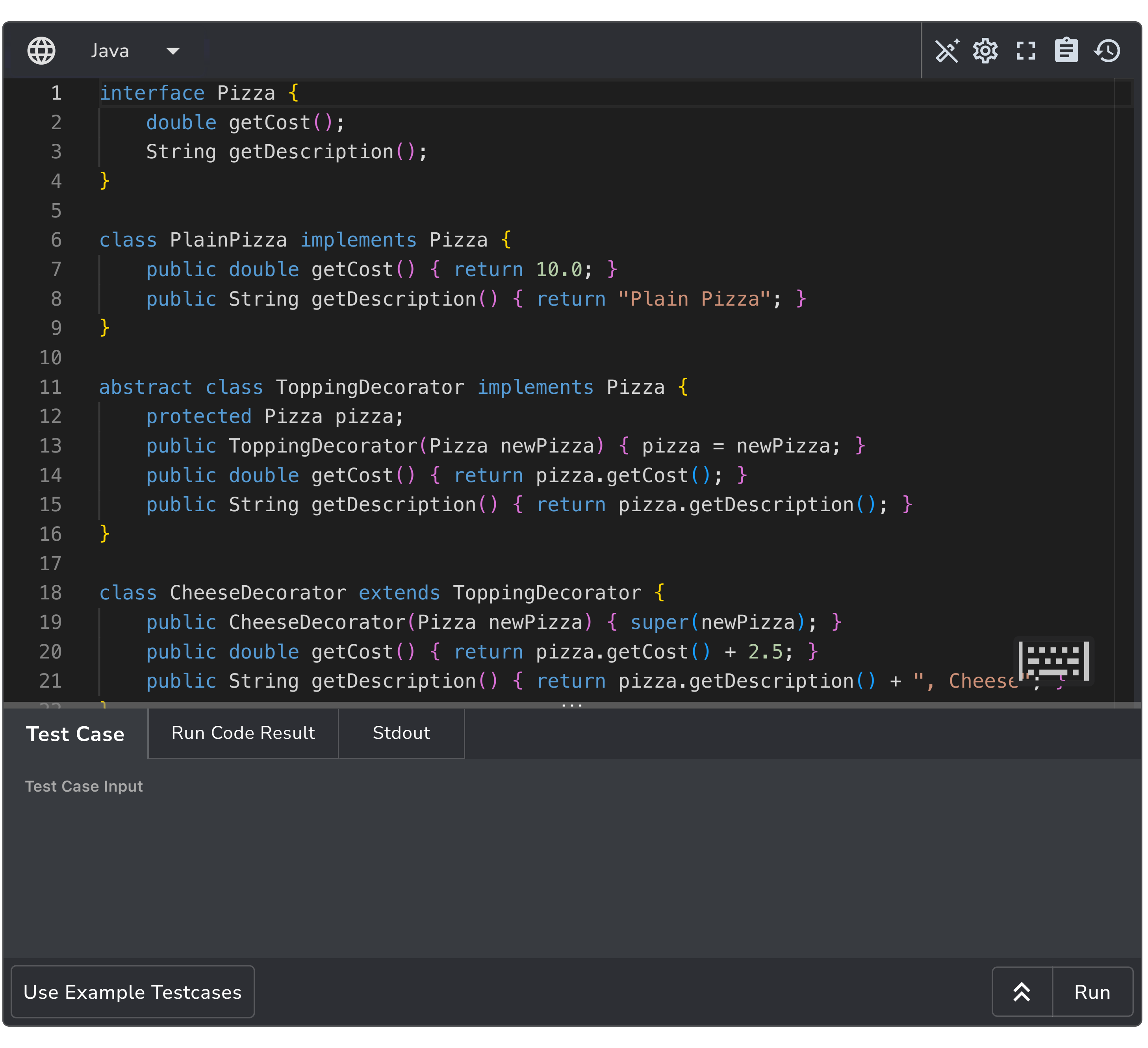
Didn't get this point? Don't worry! The pseudocode will explain you.

```
INTERFACE Pizza
    METHOD getCost()
    METHOD getDescription()

CLASS PlainPizza IMPLEMENTS Pizza
    METHOD getCost()
        RETURN base price of pizza
    METHOD getDescription()
        RETURN "Plain Pizza"

CLASS ToppingDecorator IMPLEMENTS Pizza
    PROTECTED component: Pizza
    CONSTRUCTOR ToppingDecorator(newComponent: Pizza)
        component = newComponent
    METHOD getCost()
        RETURN component.getCost()
    METHOD getDescription()
        RETURN component.getDescription()

CLASS CheeseDecorator EXTENDS ToppingDecorator
    METHOD getCost()
        RETURN component.getCost() + cost of cheese
    METHOD getDescription()
        RETURN component.getDescription() + ", Cheese"

CLASS PepperoniDecorator EXTENDS ToppingDecorator
    METHOD getCost()
        RETURN component.getCost() + cost of pepperoni
    METHOD getDescription()
        RETURN component.getDescription() + ", Pepperoni"
```

- In this example, we have a `Pizza` as an abstract class that defines the methods to get the cost and description of the Pizza. This interface is the Component Interface.
- The `PlainPizza` class is the Concrete Component that implements the `Pizza` class and gives the implementation of methods for a plain pizza.
- The `ToppingDecorator` is an abstract class that wraps a `Pizza` class object and delegates method calls to it. It also allows for the addition of some extra functionality.
- Concrete decorators like `CheeseDecorator` and `PepperoniDecorator` extend the `ToppingDecorator` and add their own cost and description to the pizza.

With the help of this pattern, you can dynamically build a pizza with different toppings that can be added one at a time without having to make a new subclass for every potential combination.

Implementation



```java
interface Pizza {
    double getCost();
    String getDescription();
}

class PlainPizza implements Pizza {
    public double getCost() { return 10.0; }
    public String getDescription() { return "Plain Pizza"; }
}

abstract class ToppingDecorator implements Pizza {
    protected Pizza pizza;
    public ToppingDecorator(Pizza newPizza) { pizza = newPizza; }
    public double getCost() { return pizza.getCost(); }
    public String getDescription() { return pizza.getDescription(); }
}

class CheeseDecorator extends ToppingDecorator {
    public CheeseDecorator(Pizza newPizza) { super(newPizza); }
    public double getCost() { return pizza.getCost() + 2.5; }
    public String getDescription() { return pizza.getDescription() + ", Cheese
```

Applications of Decorator Pattern

- **Toolkits for Graphical User Interfaces (GUIs)**:

  Adding borders, scroll bars, or color themes to individual widgets rather than creating a subclass for each combination.

- **Data Streams**:

  Wrapping data streams to include features such as formatting, encryption, buffering, and compression.

- **Web Design**:

  Adding or changing web page element behaviors dynamically on the server side before rendering, such as by adding styles or roles.

- **Tools for Reporting**:

  Adding dynamic formatting options to reports, such as headers, footers, or side notes, which can be toggled on and off as needed.

- **Game Development**:

  Adding skills, power-ups, or status changes to game characters—which can be changed or added as the game goes on.

Pros and Cons

Here's a table summarizing the pros and cons of the Decorator Pattern:

Pros	Cons
Enhanced Flexibility: It allows dynamic extension of functionality without disturbing the underlying object.	**Complexity**: Can introduce a significant amount of small classes, which can complicate the code and increase complexity.
Avoids Class Proliferation: Avoids creating too many classes for every new functionality.	**Indirection**: Adds layers of abstraction which can complicate debugging and understanding the code.
Extensibility: Complies with the Open/Closed Principle, making it easy to introduce new features without affecting existing code.	**Overuse**: Improper use can lead to systems with lots of tiny objects that could have been a simpler design.
Runtime Changes: Alterations can be done at runtime which increases the flexibility of objects used.	**Performance Issues**: Each decorator adds a level of indirection that can impact runtime performance.
Separation of Concerns: Provides better separation between the existing code and new functionalities.	**Design Complexity**: Proper design requires a deep understanding of the goal and careful planning, as misuse can lead to a confusing system architecture.

This table presents a balanced view of the Decorator Pattern's capabilities in extending functionality while highlighting potential challenges in its application.

← Previous
Composite Pattern
Next →
Facade Pattern
☐ Mark as Completed